EC4219: Software Engineering

Lecture 7 — Program Verification (2)
Inductive Assertion Method

Sunbeom So
2025 Spring

Proving Partial Correctness

- A function is **partially correct** if: (1) its precondition is satisfied at the entry (2) and its postcondition is satisfied when the function returns (if it does).
- Goal: **inductive assertion method** to prove partial correctness.
- Mathematical induction: induction on natural numbers. To show that $P(n)$ holds for all natural numbers n,
  - Base case: show $P(0)$ or $P(1)$ holds.
  - Inductive step: show $P(k) \implies P(k+1)$

  Inductive assertion method: induction on program paths. To show that F holds for every iteration of the loop,
  - Base case: show F holds at the loop entry, before executing the loop.
  - Inductive step: show F is preserved after one execution of the loop.

Steps of the Inductive Assertion Method

1. Break a function down into a finite set of basic paths.
2. Derive verification conditions (VCs) from every basic path.
3. Check the validity of VCs by an SMT solver.
4. If all of the VCs are valid, the function is partially correct.

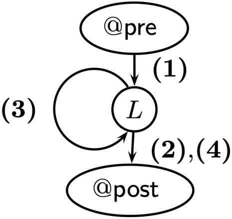
Basic Path

- A basic path is a sequence of **atomic statements** that
  - begins at the function precondition or a loop invariant, and
  - ends at a loop invariant or the function postcondition.
- Moreover, a loop invariant can only occur at the beginning or the ending of a basic path. That is, basic paths do not cross loops.

Example 1: Linear Search

@pre: $0 \leq l \land u < |a|$
@post: $rv \leftrightarrow \exists i. l \leq i \leq u \land a[i] = e$

```
bool LinearSearch (int a[], int l, int u, int e) {
  int i := l;
  while
  @L : l ≤ i ∧ (∀j.l ≤ j < i → a[j] ≠ e)
  (i ≤ u) {
    if (a[i] = e) return true;
    i := i + 1;
  }
  return false;
}
```



(1)
@pre: $0 \leq l \land u < |a|$
$i := l$;
@$L : l \leq i \land (\forall j. l \leq j < i \rightarrow a[j] \neq e)$

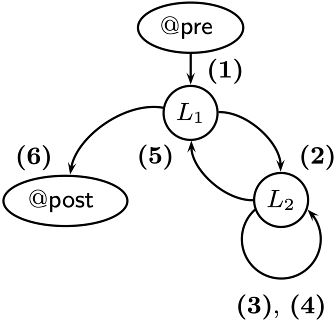
(2)
@$L : l \leq i \land (\forall j. l \leq j < i \rightarrow a[j] \neq e)$
assume $i \leq u$;
assume $a[i] = e$;
$rv :=$ true;
@post: $rv \leftrightarrow \exists i. l \leq i \leq u \land a[i] = e$

(3)
@$L : l \leq i \land (\forall j. l \leq j < i \rightarrow a[j] \neq e)$
assume $i \leq u$;
assume $a[i] \neq e$;
i := i+1;
@$L : l \leq i \land (\forall j. l \leq j < i \rightarrow a[j] \neq e)$

(4)
@$L : l \leq i \land (\forall j. l \leq j < i \rightarrow a[j] \neq e)$
assume $i > u$;
$rv :=$ false;
@post: $rv \leftrightarrow \exists i. l \leq i \leq u \land a[i] = e$

Example 2: Bubble Sort

@pre: $\top$
@post: $\textbf{sorted}(rv, 0, |rv| - 1)$
bool BubbleSort (int a[]) {
  int[] $a := a_0$;
  $@L_1 : \left\{ \begin{array}{l} -1 \leq i < |a| \\ \wedge\ \textbf{partitioned}(a, 0, i, i+1, |a|-1) \\ \wedge\ \textbf{sorted}(a, i, |a|-1) \end{array} \right\}$
  for (int $i := |a| - 1$; $i > 0$; $i := i - 1$) {
    $@L_2 : \left\{ \begin{array}{l} 1 \leq i < |a| \wedge 0 \leq j \leq i \\ \wedge\ \textbf{partitioned}(a, 0, i, i+1, |a|-1) \\ \wedge\ \textbf{partitioned}(a, 0, j-1, j, j) \\ \wedge\ \textbf{sorted}(a, i, |a|-1) \end{array} \right\}$
    for (int $j := 0$; $j < i$; $j := j + 1$) {
      if ($a[j] > a[j+1]$) {
        int $t := a[j]$;
        $a[j] := a[j+1]$;
        $a[j+1] := t$;
      }
    }
  }
  return a;
}



Example 2: Bubble Sort

(1)
@pre: $\top$
$a := a_0;$
$i := |a| - 1;$
$@L_1 : -1 \leq i < |a| \wedge \text{ partitioned}(a, 0, i, i+1, |a|-1) \wedge \text{ sorted}(a, i, |a|-1)$

(2)
$@L_1 : -1 \leq i < |a| \wedge \text{ partitioned}(a, 0, i, i+1, |a|-1) \wedge \text{ sorted}(a, i, |a|-1)$
assume $i > 0;$
$j := 0;$
$$@L_2 : \left\{ \begin{array}{l} 1 \leq i < |a| \wedge 0 \leq j \leq i \ \wedge \text{ partitioned}(a, 0, i, i+1, |a|-1) \\ \wedge \text{ partitioned}(a, 0, j-1, j, j) \ \wedge \text{ sorted}(a, i, |a|-1) \end{array} \right\}$$

(3) true branch of the inner loop
$$@L_2 : \left\{ \begin{array}{l} 1 \leq i < |a| \wedge 0 \leq j \leq i \ \wedge \text{ partitioned}(a, 0, i, i+1, |a|-1) \\ \wedge \text{ partitioned}(a, 0, j-1, j, j) \ \wedge \text{ sorted}(a, i, |a|-1) \end{array} \right\}$$
assume $j < i;$
assume $a[j] > a[j+1];$
$t := a[j];$
$a[j] := a[j+1];$
$a[j+1] := t;$
$j := j + 1;$
$$@L_2 : \left\{ \begin{array}{l} 1 \leq i < |a| \wedge 0 \leq j \leq i \ \wedge \text{ partitioned}(a, 0, i, i+1, |a|-1) \\ \wedge \text{ partitioned}(a, 0, j-1, j, j) \ \wedge \text{ sorted}(a, i, |a|-1) \end{array} \right\}$$

(4) false branch of the inner loop

$$@L_2 : \left\{ \begin{array}{l} 1 \le i < |a| \land 0 \le j \le i \ \land \ \text{partitioned}(a, 0, i, i+1, |a|-1) \\ \land \ \text{partitioned}(a, 0, j-1, j, j) \ \land \ \text{sorted}(a, i, |a|-1) \end{array} \right\}$$

assume $j < i$;

assume $a[j] \le a[j+1]$;

$j := j + 1$;

$$@L_2 : \left\{ \begin{array}{l} 1 \le i < |a| \land 0 \le j \le i \ \land \ \text{partitioned}(a, 0, i, i+1, |a|-1) \\ \land \ \text{partitioned}(a, 0, j-1, j, j) \ \land \ \text{sorted}(a, i, |a|-1) \end{array} \right\}$$

(5)

$$@L_2 : \left\{ \begin{array}{l} 1 \le i < |a| \land 0 \le j \le i \ \land \ \text{partitioned}(a, 0, i, i+1, |a|-1) \\ \land \ \text{partitioned}(a, 0, j-1, j, j) \ \land \ \text{sorted}(a, i, |a|-1) \end{array} \right\}$$

assume $j \ge i$;

$i := i - 1$;

$@L_1 : -1 \le i < |a| \land \ \text{partitioned}(a, 0, i, i+1, |a|-1) \land \ \text{sorted}(a, i, |a|-1)$

(6)

$@L_1 : -1 \le i < |a| \land \ \text{partitioned}(a, 0, i, i+1, |a|-1) \land \ \text{sorted}(a, i, |a|-1)$

assume $i \le 0$;

$rv := a$;

@post: $\text{sorted}(rv, 0, |rv|-1)$

Basic Path: Function Calls

- Like loops, recursive function calls may create an unbounded number of paths. Thus, simply inlining function calls would not work in general.
- Like a loop invariant, a function postcondition summarizes the effects of the function invocation.
- Warning: the function postcondition holds only when the precondition is satisfied on the entry. Thus, we should generate extra basic paths to verify the precondition.

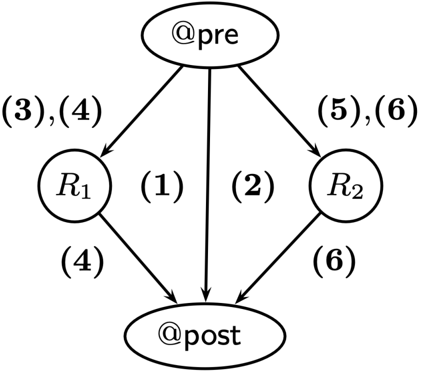
Example 3: Binary Search

```
@pre: 0 ≤ l ∧ u < |a| ∧ sorted(a, l, u)
@post: rv ↔ ∃i.l ≤ i ≤ u ∧ a[i] = e
bool BinarySearch (int a[], int l, int u, int e) {
  if (l > u) return false;
  else {
    int m := (l + u) div 2;
    if (a[m] = e) return true;
    else if (a[m] < e)
      @R₁ : 0 ≤ m + 1 ∧ u < |a| ∧ sorted(a, m + 1, u)
      return BinarySearch (a, m + 1, u, e)
    else
      @R₂ : 0 ≤ l ∧ m − 1 < |a| ∧ sorted(a, l, m − 1)
      return BinarySearch (a, l, m − 1, e)
  }
}
```

$@R_1$ and $@R_2$ are the **precondition assertions** generated by replacing formal input parameters with arguments from @pre $: F[a, l, u, e]$.

Example 3: Binary Search

Visualization of the basic paths:



Example 3: Binary Search

(1)

@pre: $0 \leq l \wedge u < |a| \wedge \textbf{sorted}(a, l, u)$
assume $l > u$;
$rv :=$ false;
@post: $rv \leftrightarrow \exists i. l \leq i \leq u \wedge a[i] = e$

(2)

@pre: $0 \leq l \wedge u < |a| \wedge \textbf{sorted}(a, l, u)$
assume $l \leq u$;
$m := (l + u)$ div 2;
assume $a[m] = e$;
$rv :=$ true;
@post: $rv \leftrightarrow \exists i. l \leq i \leq u \wedge a[i] = e$

Example 3: Binary Search

Basic paths to verify the preconditions hold or not at each call site.

$\mathbf{(3)}$

$@\text{pre: } 0 \le l \wedge u < |a| \wedge \textsf{sorted}(a, l, u)$

assume $l \le u$;

$m := (l + u) \text{ div } 2$;

assume $a[m] \ne e$;

assume $a[m] < e$;

$@R_1 : 0 \le m + 1 \wedge u < |a| \wedge \textsf{sorted}(a, m + 1, u)$

$\mathbf{(5)}$

$@\text{pre: } 0 \le l \wedge u < |a| \wedge \textsf{sorted}(a, l, u)$

assume $l \le u$;

$m := (l + u) \text{ div } 2$;

assume $a[m] \ne e$;

assume $a[m] \ge e$;

$@R_2 : 0 \le l \wedge m - 1 < |a| \wedge \textsf{sorted}(a, l, m - 1)$

Example 3: Binary Search

The basic path passing through the function call at line 10.

$$\begin{array}{l}
(4) \\
\text{@pre: } 0 \leq l \land u < |a| \land \textbf{sorted}(a, l, u) \\
\text{assume } l \leq u; \\
m := (l + u) \text{ div } 2; \\
\text{assume } a[m] \neq e; \\
\text{assume } a[m] < e; \\
\text{assume } v' \leftrightarrow \exists i.\ m + 1 \leq i \leq u \land a[i] = e; \\
rv := v'; \\
\text{@post: } rv \leftrightarrow \exists i. l \leq i \leq u \land a[i] = e
\end{array}$$

Let $G[a, l, u, e, rv]$ be the postcondition. The second and third statements from the end are obtained by converting line 10 into

$$\text{assume } G[a, m + 1, u, e, v']; \quad rv := v'$$

Since we verify the precondition from (3), we can use $G[a, m + 1, u, e, v']$ as a summary of the function call at line 10.

Example 3: Binary Search

The basic path passing through the function call at line 13.

$$
\begin{array}{l}
\textbf{(6)} \\
\text{@pre: } 0 \le l \land u < |a| \land \textbf{sorted}(a, l, u) \\
\text{assume } l \le u; \\
m := (l + u) \text{ div } 2; \\
\text{assume } a[m] \ne e; \\
\text{assume } a[m] \ge e; \\
\text{assume } v' \leftrightarrow \exists i.\ l \le i \le m - 1 \land a[i] = e; \\
rv := v'; \\
\text{@post: } rv \leftrightarrow \exists i. l \le i \le u \land a[i] = e
\end{array}
$$

Generating VCs

- To generate VCs for each basic path, we need to express the effects of program statements in terms of FOL formulas.
- The **weakest precondition** predicate transformer is the mechanism; it generates the most general one among valid preconditions.

$$\textbf{wp} : \textbf{FOL} \times \textbf{stmts} \to \textbf{FOL}$$

What is the precondition that must hold before the statement to ensure that the postcondition holds afterwards?

- $\{ \qquad\qquad \}$ `x:=x+1` $\{x > 0\}$
- $\{ \qquad\qquad \}$ `y:=2*y` $\{y < 5\}$
- $\{ \qquad\qquad \}$ `x:=x+y` $\{y > x\}$
- $\{ \qquad \}$ `assume` $(a \leq 5)$ $\{a \leq 5\}$
- $\{ \qquad\qquad\qquad \}$ `assume` $(a \leq b)$ $\{a \leq 5\}$

Generating VCs

- To generate VCs for each basic path, we need to express the effects of program statements in terms of FOL formulas.
- The **weakest precondition** predicate transformer is the mechanism; it generates the most general one among valid preconditions.

$$\textbf{wp} : \textbf{FOL} \times \textbf{stmts} \to \textbf{FOL}$$

What is the precondition that must hold before the statement to ensure that the postcondition holds afterwards?

- $\{x + 1 > 0\}$ `x:=x+1` $\{x > 0\}$
- $\{2 * y < 5\}$ `y:=2*y` $\{y < 5\}$
- $\{y > x + y\}$ `x:=x+y` $\{y > x\}$
- $\{true\}$ `assume` $(a \leq 5)$ $\{a \leq 5\}$
- $\{a \leq b \to a \leq 5\}$ `assume` $(a \leq b)$ $\{a \leq 5\}$

Weakest Precondition Predicate Transformer

Weakest precondition $\textbf{wp}(F, S)$ for statements S of basic paths:

- Assumption: What must hold before the $\texttt{assume}(c)$ is executed to ensure that F holds afterwards? If $c \to F$ holds before, then satisfying c guarantees that F holds afterwards:

$$\textbf{wp}\ (F, \texttt{assume}(c)) \iff c \to F.$$

- Assignment: What must hold before the statement $v := e$ is executed to ensure that F holds afterward? If $F[e/v]$ holds before, then assigning e to v makes F holds afterward:

$$\textbf{wp}(F, v := e) \iff F[e/v]$$

- For a sequence of statements $S_1; \cdots ; S_n$, compute backwardly.

$$\textbf{wp}(F, S_1; \cdots ; S_n) \iff \textbf{wp}(\textbf{wp}(F, S_n), S_1; \cdots , S_{n-1})$$

Verification Condition

The verification condition of a basic path

$$\begin{array}{c} @F \\ S_1; \\ \vdots \\ S_n; \\ @G \end{array}$$

is

$$F \rightarrow \textbf{wp}(G, S_1; \cdots ; S_n)$$

The verification condition is also denoted by the Hoare triple

$$\{F\} S_1; \cdots ; S_n \{G\}$$

Example 1

Consider the basic path

$$
\begin{aligned}
&@x \geq 0 \\
&x := x + 1; \\
&@x \geq 1
\end{aligned}
$$

and its corresponding Hoare triple $\{x \geq 0\}\ x := x + 1\ \{x \geq 1\}$. The verification proceeds as follows:

$$
\begin{array}{rl}
& x \geq 0 \rightarrow \textbf{wp}(x \geq 1, x := x + 1) \\
\iff & x \geq 0 \rightarrow x + 1 \geq 1 \\
\iff & true
\end{array}
$$

which is valid.

Example 2

Consider the basic path (2) from the LinearSearch example.

$$
\begin{array}{l}
@L : F : l \leq i \land (\forall j. l \leq j < i \rightarrow a[j] \neq e) \\
S_1 : \texttt{assume } i \leq u; \\
S_2 : \texttt{assume } a[i] = e; \\
S_3 : rv := \text{true} \\
@\text{post } G : rv \leftrightarrow \exists i. l \leq i \leq u \land a[i] = e
\end{array}
$$

Its verification condition is

$$
\begin{array}{ll}
 & F \rightarrow \mathbf{wp}(G, S_1; S_2; S_3) \\
\Leftrightarrow & F \rightarrow \mathbf{wp}(\mathbf{wp}(rv \leftrightarrow \exists i. l \leq i \leq u \land a[i] = e, rv := \text{true}), S_1; S_2) \\
\Leftrightarrow & F \rightarrow \mathbf{wp}(true \leftrightarrow \exists i. l \leq i \leq u \land a[i] = e, S_1; S_2) \\
\Leftrightarrow & F \rightarrow \mathbf{wp}(\exists i. l \leq i \leq u \land a[i] = e, S_1; S_2) \\
\Leftrightarrow & F \rightarrow \mathbf{wp}(\mathbf{wp}(\exists i. l \leq i \leq u \land a[i] = e, \texttt{assume } a[i] = e), S_1) \\
\Leftrightarrow & F \rightarrow \mathbf{wp}(a[i] = e \rightarrow \exists i. l \leq i \leq u \land a[i] = e, S_1) \\
\Leftrightarrow & F \rightarrow (i \leq u \rightarrow (a[i] = e \rightarrow \exists i. l \leq i \leq u \land a[i] = e))
\end{array}
$$

Partial Correctness

- So far: how to verify the specification for each basic path.
- If the verification succeeds in every basic path, the program is partially correct.

Theorem

Given a program P, if for every basic path

$$\begin{array}{c} @F \\ S_1; \\ \vdots; \\ S_n; \\ @G \end{array}$$

the verification condition $\{F\}S_1; \cdots ; S_n\{G\}$ is valid, then the program obeys its specification.

Handling Arrays (1st attempt)

- How can we model the effect of array element assignments $x[e_1] := e_2$? Is the following definition correct?

$$\mathbf{wp}(F, x[e_1] := e_2) \iff F[e_2/x[e_1]]$$

Handling Arrays (1st attempt)

- How can we model the effect of array element assignments $x[e_1] := e_2$? Is the following definition correct?

$$\mathbf{wp}(F, x[e_1] := e_2) \iff F[e_2/x[e_1]]$$

- Incorrect! Counterexample?

Handling Arrays (1st attempt)

- How can we model the effect of array element assignments $x[e_1] := e_2$? Is the following definition correct?

$$\mathbf{wp}(F, x[e_1] := e_2) \iff F[e_2/x[e_1]]$$

- Incorrect! Counterexample?

$$\{i = 1\}\ x[i] := 3; x[1] := 2\ \{x[i] = 3\}$$

Handling Arrays (1st attempt)

- How can we model the effect of array element assignments $x[e_1] := e_2$? Is the following definition correct?

$$\textsf{wp}(F, x[e_1] := e_2) \iff F[e_2/x[e_1]]$$

- Incorrect! Counterexample?

$$\begin{array}{rl} & \{i = 1\}\ x[i] := 3; x[1] := 2\ \{x[i] = 3\} \\ \iff & i = 1 \rightarrow \textsf{wp}(x[i] = 3, x[i] := 3; x[1] := 2) \\ \iff & i = 1 \rightarrow \textsf{wp}(\textsf{wp}(x[i] = 3, x[1] := 2), x[i] := 3) \\ \iff & i = 1 \rightarrow \textsf{wp}(x[i] = 3, x[i] := 3) \\ \iff & i = 1 \rightarrow true \end{array}$$

Although the Hoare triple is invalid, the VC is proven (unsound proof)!

Handling Arrays (Corrected)

- The correct transformer:

$$\mathbf{wp}(F, x[e_1] := e_2) \iff F[x\langle e_1 \triangleleft e_2\rangle / x]$$

  The idea is to replace x with a variable that is the same as x, with a potential exception at index e_1.

- Using the corrected transformer, the VC is invalid as desired.

$$\begin{array}{rl} & \{i = 1\}\ x[i] := 3; x[1] := 2\ \{x[i] = 3\} \\ \iff & i = 1 \to \mathbf{wp}(x[i] = 3, x[i] := 3; x[1] := 2) \\ \iff & i = 1 \to \mathbf{wp}(\mathbf{wp}(x[i] = 3, x[1] := 2), x[i] := 3) \\ \iff & i = 1 \to \mathbf{wp}(x\langle 1 \triangleleft 2\rangle[i] = 3, x[i] := 3) \\ \iff & i = 1 \to (x\langle i \triangleleft 3\rangle)\langle 1 \triangleleft 2\rangle[i] = 3 \end{array}$$

Strongest Postcondition Predicate Transformer

Unlike **wp**, **sp** generates VC forwardly.

- **sp** $(F, \texttt{assume}(c)) \iff c \land F$.
- **sp**$(F, v := e) \iff F[v'/v] \land v = e[v'/v]$, where v' is a fresh variable that represents v before updated.
- For a sequence of statements $S_1; \cdots; S_n$, compute forwardly.

$$\textbf{sp}(F, S_1; \cdots; S_n) \iff \textbf{sp}(\textbf{sp}(F, S_1), S_2; \cdots, S_n)$$

The Hoare triple

$$\{F\}S_1; \cdots; S_n\{G\}$$

can be verified by checking the validity of

$$\textbf{sp}(F, S_1; \cdots; S_n) \rightarrow G$$

Summary

We explored the **inductive assertion method** for proving **partial correctness**.

1. Break a function down into a set of basic paths.
2. Derive verification conditions (VCs) from every basic path.
3. Check the validity of VCs by an SMT solver.
4. If all of the VCs are valid, the function is partially correct.

The method can be automated, if proper loop invariants are given. Automatically generating loop invariants, however, is not an easy task and remains an active research area.